

Autoescalamamento Horizontal de Pods (HPA) em Kubernetes: um estudo comparativo entre Minikube e Amazon EKS

Murilo Gabriel Moraes de Azevedo, Bruno Valle Martins e Pedro Remus de Ávila
Escola Politécnica da Universidade de São Paulo
PSI5120 — Tópicos em Computação em Nuvem (2026)
Nº USP: 13782776, 13681036, 13682486

Resumo—Este trabalho projeta, implanta e testa um servidor web em Kubernetes configurado com *Horizontal Pod Autoscaler* (HPA), comparando duas plataformas: um cluster local com Minikube e um cluster gerenciado na nuvem com Amazon EKS. A mesma aplicação, os mesmos manifestos e o mesmo protocolo de teste de estresse são aplicados às duas implantações. Na implantação local, o HPA reagiu a uma carga sintética de CPU escalando o número de réplicas de 1 para o teto de 10 em cerca de três minutos (transições 1→4→8→10) e, após o término da carga, reduziu de volta a uma réplica respeitando a janela de estabilização de 300 s. Na nuvem, o mesmo protocolo em Amazon EKS com dois nós reproduziu a escala (1 a 10) de forma ainda mais rápida (cerca de 1,5 min) e distribuiu as réplicas igualmente entre os nós (5/5), diferença ausente no nó único local. Discutem-se tempo de reação, consumo de recursos, custos e facilidade de gerenciamento, destacando as diferenças arquiteturais entre executar o autoescalamamento em um nó único local e em um cluster multi-nó gerenciado. Conclui-se que o mecanismo de HPA é portátil entre as plataformas, mas o contexto operacional — provedor de métricas, origem da imagem, distribuição entre nós, custo e a necessidade de escalar também a capacidade de nós — é o que distingue um laboratório local de uma operação elástica em nuvem.

Index Terms—Kubernetes, autoescalamamento, HPA, Minikube, Amazon EKS, computação em nuvem, elasticidade, teste de estresse, contêineres.

I. INTRODUÇÃO

A elasticidade — a capacidade de ajustar recursos computacionais à demanda de forma automática — é um dos princípios definidores da computação em nuvem. Sistemas subprovisionados degradam sob picos de carga, violando acordos de nível de serviço; sistemas superprovisionados desperdiçam recurso e dinheiro durante os vales de utilização. O equilíbrio dinâmico entre esses extremos é precisamente o que os mecanismos de autoescalamamento buscam oferecer, substituindo o dimensionamento estático e manual por um controle contínuo guiado por métricas.

Em plataformas de orquestração de contêineres como o Kubernetes, a elasticidade se manifesta em diferentes níveis. No nível da aplicação, o *Horizontal Pod Autoscaler* (HPA) [1] ajusta o número de réplicas (*Pods*) de uma carga de trabalho com base em métricas observadas, como a utilização de CPU.

No nível do *Pod*, o *Vertical Pod Autoscaler* ajusta os recursos solicitados por contêiner. No nível da infraestrutura, o *Cluster Autoscaler* e o *Karpenter* ajustam o número de nós. Este trabalho concentra-se no primeiro nível, o mais diretamente ligado à noção de escalar horizontalmente uma aplicação web.

O objetivo é **projetar, implantar e testar** um servidor web sob HPA em duas plataformas distintas e obrigatórias: (i) um cluster **local** com Minikube e (ii) um cluster **gerenciado na nuvem** com Amazon EKS [2]. A hipótese de trabalho é que o *mecanismo* de autoescalamamento é idêntico nas duas plataformas — pois o HPA é um recurso nativo do Kubernetes —, mas o *contexto operacional* (instalação do provedor de métricas, origem da imagem, distribuição entre nós, custo e esforço de gerenciamento) difere de forma relevante e determina a aplicabilidade de cada ambiente.

As contribuições deste artigo são: (a) uma implementação **reprodutível** de um servidor web sob HPA, com manifestos comentados, imagem própria e *scripts* de automação versionados em repositório público; (b) a **caracterização empírica** da dinâmica do HPA sob teste de estresse na implantação local, incluindo os tempos de reação de subida e descida e o consumo por réplica; e (c) uma **análise comparativa** entre a execução local e a gerenciada na nuvem quanto a tempo de reação, consumo, custo e gerenciamento. O restante do artigo detalha os trabalhos relacionados (Seção II), a fundamentação (Seção III), a arquitetura (Seção IV), a metodologia (Seção V), os resultados (Seção VI), a análise comparativa (Seção VII), a discussão (Seção VIII), as ameaças à validade (Seção IX) e as conclusões (Seção X).

II. TRABALHOS RELACIONADOS

O autoescalamamento em nuvem é tema consolidado. A documentação oficial do Kubernetes [1] apresenta o *walkthrough* canônico do HPA, no qual uma aplicação PHP-Apache limitada por CPU é escalada sob carga; este trabalho adota a mesma classe de aplicação, mas em um estudo *comparativo* entre plataformas local e gerenciada. A documentação da AWS [2] descreve o HPA no contexto do EKS, ressaltando a necessidade de instalar o *Metrics Server* e a interação com

a capacidade de nós. A literatura de sistemas distingue o autoescalamento *reativo* (baseado em limiares sobre métricas correntes, como o HPA por CPU) do *preditivo* (baseado em previsão de demanda); o presente estudo situa-se no primeiro grupo. Em relação a esses materiais, a contribuição aqui é metodológica e empírica: aplicar um protocolo idêntico de estresse a duas plataformas e caracterizar quantitativamente a resposta observada, tornando explícitas as diferenças de *contexto* que não aparecem quando se estuda uma única plataforma.

III. FUNDAMENTAÇÃO

A. Kubernetes: objetos relevantes

O Kubernetes organiza aplicações em objetos declarativos. Um Pod é a menor unidade implantável; um Deployment declara o estado desejado de uma carga (imagem, réplicas, recursos) e o mantém por meio de um ReplicaSet; um Service fornece um ponto de acesso estável a um conjunto de Pods. O HorizontalPodAutoscaler atua sobre o Deployment, alterando seu número de réplicas. O plano de controle (servidor de API, *scheduler*, controladores e *etcd*) reconcilia continuamente o estado observado com o desejado; em cada nó, o *kubelet* materializa os Pods atribuídos.

B. Horizontal Pod Autoscaler

O HPA é um controlador que executa em laço fechado. Periodicamente (por padrão, a cada 15 s), consulta a métrica-alvo de todos os Pods da carga, calcula a utilização média e determina o número desejado de réplicas pela relação

$$R_{des} = \left\lceil R_{atual} \times \frac{M_{atual}}{M_{alvo}} \right\rceil, \quad (1)$$

onde R é o número de réplicas e M a métrica — neste trabalho, a utilização média de CPU como percentual do `requests.cpu` declarado no contêiner. O resultado é limitado por `minReplicas` e `maxReplicas`. Uma banda de tolerância (por padrão, 10%) evita reações a flutuações pequenas em torno do alvo.

Para evitar oscilação (*flapping*), o HPA aplica **janelas de estabilização** distintas para subida e descida. A subida costuma ser imediata; a descida, por padrão, observa uma janela de 300 s, durante a qual o controlador não reduz abaixo do maior número de réplicas recomendado no período. Há ainda *políticas de escala* que limitam a taxa de variação por intervalo (por exemplo, no máximo dobrar as réplicas ou adicionar quatro Pods). Esses parâmetros explicam por que a resposta observada avança em passos e não em um único salto.

C. Taxonomia do autoescalamento

Convém distinguir três mecanismos complementares. O **HPA** escala o número de réplicas (horizontal, no nível da aplicação). O **VPA** (*Vertical Pod Autoscaler*) ajusta os recursos solicitados por contêiner (vertical). O **Cluster Autoscaler**/Karpenter ajusta o número de nós. Um ponto central deste trabalho é que o HPA, isoladamente, escala *Pods*, não nós: se o teto de réplicas exigir mais capacidade do que o cluster

```
FROM php:8.2-apache
COPY index.php /var/www/html/index.php
EXPOSE 80
# a imagem base já inicia o Apache em foreground
```

Figura 1. Dockerfile do servidor web (comentado no repositório).

possui, os Pods excedentes permanecem Pending até que a capacidade de nós aumente — o que o HPA não faz por si só.

D. Provedor de métricas: Metrics Server

O HPA por CPU depende de um provedor de métricas. O *Metrics Server* coleta o uso de CPU e memória de cada Pod a partir dos *kubelets* e o expõe pela *Metrics API*. É condição necessária: sem ele, o HPA reporta a métrica como `<unknown>` e não escala. Há uma latência inicial até o *Metrics Server* publicar as primeiras amostras após a criação dos Pods.

E. Minikube e Amazon EKS

O **Minikube** cria um cluster Kubernetes **local de nó único**, adequado a estudo e desenvolvimento; o *Metrics Server* é habilitado por um *addon* de um comando, e imagens locais podem ser injetadas com `minikube image load`. O **Amazon EKS** é um serviço **gerenciado** em que a AWS opera o plano de controle e o usuário provê capacidade por meio de um *managed node group* de instâncias EC2. No EKS, o *Metrics Server* é instalado manualmente por manifesto e as imagens vêm de um *registry* (Amazon ECR ou Docker Hub). Essas diferenças não alteram o *mecanismo* do HPA, mas mudam o *contexto* de operação.

IV. ARQUITETURA E TOPOLOGIA

A aplicação é um servidor web (`php:8.2-apache`) cujo `index.php` executa, a cada requisição, um laço de cálculo de raiz quadrada que consome CPU de forma mensurável — a mesma classe de aplicação do *walkthrough* do HPA [1]. Escolheu-se uma carga **limitada por CPU** justamente porque a métrica de autoescalamento é a utilização de CPU. A imagem é construída a partir de um Dockerfile próprio (Figura 1) e versionada no repositório.

A Tabela I lista os objetos Kubernetes da solução e seu papel. A carga é declarada por um Deployment com `requests.cpu = 200m` e `limits.cpu = 500m` (Figura 2); definir `requests` é **obrigatório**, pois o alvo de utilização do HPA é medido como percentual desse valor. Um Service ClusterIP dá acesso estável ao conjunto de réplicas, e o HPA (Figura 3) mantém a utilização média de CPU em 50% do `requests`, entre 1 e 10 réplicas. A Figura 4 resume a topologia comum às duas implantações.

A diferença de topologia está no substrato. No **Minikube** há um **único nó** — todas as réplicas residem nele — e a imagem é injetada com `minikube image load`. No **EKS** há um *managed node group* com **múltiplos nós EC2** sobre os quais os Pods podem se distribuir, e a imagem vem de um *registry*.

Tabela I
OBJETOS KUBERNETES DA SOLUÇÃO

Objeto	Papel
Namespace avl	escopo lógico e limpeza
Deployment web	servidor web; alvo do HPA; declara requests/limits de CPU
Service web	acesso interno estável (ClusterIP) ao conjunto de Pods
HPA web	mantém CPU média em 50%; 1 a 10 réplicas
Deployment load-generator	teste de estresse (requisições HTTP em laço)
Metrics Server	fornece a métrica de CPU ao HPA

```
spec:
  replicas: 1          # o HPA passa a controlar
  template:
    spec:
      containers:
      - name: web
        image: avl-webcpu:v1
        resources:
          requests: { cpu: 200m, memory: 64Mi }
          limits:   { cpu: 500m, memory: 128Mi }
```

Figura 2. Trecho do Deployment (requests/limits de CPU).

```
kind: HorizontalPodAutoscaler # autoscaling/v2
spec:
  scaleTargetRef: { kind: Deployment, name: web }
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization,
                averageUtilization: 50 }
```

Figura 3. Manifesto do HPA usado nas duas implantações.

```
[ Gerador de carga ] --HTTP--> [ Service web ]
                              |
                              v
          [ Deployment web ] (1..10 Pods)
            ^ ajusta replicas
            |
[ HPA: CPU media 50% ] <-- [ Metrics Server ]
```

Figura 4. Topologia comum. O HPA lê a CPU dos Pods pelo Metrics Server e ajusta as réplicas do Deployment; o gerador de carga eleva a CPU e dispara o escalamento.

V. METODOLOGIA

O mesmo protocolo foi definido para as duas plataformas, garantindo comparabilidade:

- 1) provisionar o cluster e **garantir o Metrics Server ativo**;
- 2) disponibilizar a imagem do servidor web (carga local na plataforma local; *registry* na nuvem);
- 3) aplicar Deployment, Service e HPA;
- 4) registrar o **baseline** (1 réplica, CPU ociosa);
- 5) iniciar o **teste de estresse**, aplicando um Deployment gerador de carga com cinco réplicas de um cliente que

```
# gerador de carga (Deployment, 5 replicas):
while true; do
  wget -q -O- http://web > /dev/null
done
```

Figura 5. Laço do gerador de carga (teste de estresse).

Tabela II
LINHA DO TEMPO DO HPA NA IMPLANTAÇÃO MINIKUBE

Momento	CPU (atual/alvo)	Réplicas
Baseline	0 % / 50 %	1
Início da carga (+0 s)	0 % / 50 %	1
+60 s	161 % / 50 %	1 → 4
+2 min	183 % / 50 %	8
+3 min	106 % / 50 %	10 (máx.)
Fim da carga (+0 s)	106 % / 50 %	10
+90 s	0 % / 50 %	10
+5 min (janela)	0 % / 50 %	10 → 1

envia requisições HTTP em laço ao Service;

- 6) observar o *scale up* (utilização de CPU e número de réplicas ao longo do tempo);
- 7) encerrar a carga e observar o *scale down* até o retorno ao estado inicial.

O gerador de carga (Figura 5) foi dimensionado para saturar o alvo de 50 % de CPU: cinco clientes concorrentes mantêm requisições contínuas, cada uma disparando o laço de CPU do servidor. As evidências — estado antes, durante e após a carga; métricas; e eventos do controlador — foram coletadas com `kubectl get hpa`, `kubectl get pods`, `kubectl top pods` e `kubectl describe hpa`, com carimbos de tempo a cada 20s para reconstruir a linha do tempo. Todo o material (manifestos, *scripts* e saídas) está versionado em repositório público, permitindo reprodução por qualquer integrante do grupo.

VI. RESULTADOS

A. Implantação A — Minikube (local)

O cluster local (Minikube v1.38, Kubernetes v1.35, *driver* Docker, nó único, quatro vCPUs disponíveis) foi provisionado e o *Metrics Server* habilitado por *addon*. No **baseline**, o HPA reportou `cpu: 0%/50%` com **1 réplica** consumindo cerca de 1 m de CPU — estado ocioso.

Ao iniciar o teste de estresse, a utilização de CPU subiu rapidamente e o HPA executou uma sequência de reescalamentos. A Tabela II resume a linha do tempo observada.

Os eventos do controlador (`kubectl describe hpa`) confirmam a progressão: `SuccessfulRescale` para **4**, **8** e **10** réplicas (*“cpu resource utilization above target”*) e, após o fim da carga, `SuccessfulRescale` para **1** (*“All metrics below target”*). A sequência 1→4→8→10 é coerente com a Equação 1 combinada às políticas de taxa de escala: a partir de uma réplica a 161 %, o número desejado calculado é alto, mas o controlador avança em passos limitados por intervalo até atingir o teto de 10.

No pico, as dez réplicas apresentaram consumo **homogêneo** — aproximadamente 198 a 229 m de CPU cada (Tabela III),

Tabela III
CONSUMO POR RÉPLICA NO PICO (AMOSTRA)

Pod	CPU (milicores)
web-...-n2glk	198 m
web-...-mhp2g	206 m
web-...-2mcnq	213 m
web-...-6v458	222 m
web-...-mmsg9	229 m

próximo do `requests` de 200m —, todas agendadas no **mesmo nó**, característica do Minikube de nó único. A utilização média reportada no pico (106%) ainda acima do alvo indica que, se houvesse teto maior, o HPA teria continuado a escalar; o valor foi contido pelo `maxReplicas` = 10.

B. Dinâmica temporal

A dinâmica observada é **assimétrica**. O *scale up* foi **agressivo**: a primeira reação ocorreu em cerca de 60s após o início da carga (limitada, no primeiro minuto, pela latência inicial do *Metrics Server*, que emitiu avisos `FailedGetResourceMetric` até publicar as primeiras amostras) e o teto de 10 réplicas foi atingido em cerca de 3 min. O *scale down* foi **conservador**: encerrada a carga, a CPU caiu a 0% em cerca de 90s, mas as réplicas **permaneceram em 10 durante a janela de estabilização de 300s**, reduzindo a 1 apenas após esse período. Esse comportamento é intencional — prioriza disponibilidade na subida e evita oscilação na descida — e foi reproduzido fielmente pela configuração padrão do controlador.

C. Implantação B — Amazon EKS (nuvem)

A implantação em EKS reutilizou os mesmos manifestos e o mesmo protocolo, efetivando três diferenças operacionais: o *Metrics Server* foi instalado por manifesto oficial; a imagem foi obtida de um *registry* público; e o *managed node group* foi provisionado com **dois nós** `t3.medium` (Amazon Linux 2023, Kubernetes v1.36) em zonas de disponibilidade distintas.

No **baseline**, o HPA reportou `cpu: 0%/50%` com 1 réplica. Sob o teste de estresse, o HPA escalou até o teto de 10 réplicas em cerca de **1,5 min** — mais rápido que no ambiente local —, com as transições 1→3→5→9→10. A Tabela IV resume a linha do tempo, e os eventos `SuccessfulRescale` confirmam a progressão para 3, 5, 9 e 10 réplicas e, após o fim da carga, para 2 e 1 (“*All metrics below target*”).

O ponto que **distingue o EKS do Minikube** é a distribuição: no pico, as dez réplicas ficaram divididas **5/5 entre os dois nós EC2** (Tabela V), evidenciando o balanceamento de Pods entre nós que um cluster de nó único não permite. O consumo por réplica ficou entre 116 e 234m de CPU. A dinâmica de descida foi novamente **conservadora**: a CPU caiu a 0% em cerca de 1,5 min, mas as réplicas permaneceram em 10 durante a janela de estabilização de 300s antes de retornarem a 1 — o mesmo comportamento assimétrico observado no Minikube, confirmando que ele é uma propriedade do controlador, não da plataforma.

Tabela IV
LINHA DO TEMPO DO HPA NA IMPLANTAÇÃO AMAZON EKS

Momento	CPU (atual/alvo)	Réplicas
Baseline	0% / 50%	1
Início da carga (+0 s)	0% / 50%	1
+20 s	113% / 50%	1 → 3
+40 s	139% / 50%	5
+60 s	122% / 50%	9
+90 s	98% / 50%	10 (máx.)
Fim da carga (+0 s)	97% / 50%	10
+90 s	0% / 50%	10
+5 min (janela)	0% / 50%	10 → 2 → 1

Tabela V
DISTRIBUIÇÃO DAS 10 RÉPLICAS ENTRE OS NÓS (EKS, PICO)

Nó EC2	Réplicas
ip-192-168-76-209 (AZ 1)	5
ip-192-168-185-242 (AZ 2)	5

Tabela VI
COMPARAÇÃO QUALITATIVA MINIKUBE VS. AMAZON EKS

Critério	Minikube (local)	Amazon EKS (nuvem)
Mecanismo de HPA	idêntico (nativo do Kubernetes)	idêntico
Metrics Server	<i>addon</i> (um comando)	manifesto manual
Origem da imagem	minikube image load	<i>registry</i> (ECR/Docker Hub)
Topologia	nó único; réplicas no mesmo nó	multi-nó; réplicas distribuídas
Escala de nós	não aplicável	requer Cluster Autoscaler/Karpenter
Custo	≈ zero (recursos locais)	control plane + EC2 + EBS (por hora)
Gerenciamento	simples; efêmero	plano de controle gerenciado; mais configuração
Uso indicado	estudo, desenvolvimento	produção, elasticidade real

VII. ANÁLISE COMPARATIVA

A Tabela VI sintetiza a comparação segundo os critérios solicitados. Em seguida, discutem-se os pontos de maior impacto.

Tempo de reação. O mecanismo do HPA é o mesmo; empiricamente, o EKS atingiu o teto de 10 réplicas em cerca de 1,5 min, contra cerca de 3 min no Minikube. A diferença não vem do mecanismo (o período de sincronização é 15s em ambos), mas do contexto: com dois nós, o EKS dispõe de mais CPU agregada, de modo que a utilização média permanece alta por mais tempo e o controlador recomenda mais réplicas em cada avaliação; no nó único, a competição por CPU e o eventual *throttling* moderam a métrica.

Consumo de recursos. No Minikube, as dez réplicas competem pela CPU de um único nó; se o nó saturar, os Pods sofrem *throttling* e a utilização observada deixa de crescer. No EKS multi-nó, as réplicas se distribuem, reduzindo

Tabela VII
COMPARAÇÃO EMPÍRICA DAS DUAS EXECUÇÕES REAIS

Métrica	Minikube	Amazon EKS
Réplicas (mín.–máx.)	1–10	1–10
Transições de subida	1,4,8,10	1,3,5,9,10
Tempo até o teto	≈3 min	≈1,5 min
CPU no pico (média)	106 %	85–100 %
CPU/réplica no pico	198–229 m	116–234 m
Nós	1	2
Distribuição no pico	10 no mesmo nó	5/5 entre 2 nós
Janela de descida	300 s	300 s
Custo do experimento	≈0	≈US\$ 0,50

a contenção por nó e aproximando a utilização medida da demanda real.

Custo. O Minikube é essencialmente **gratuito** (recursos locais), ideal para validar o mecanismo. O EKS incorre em custo por hora do plano de controle e das instâncias EC2 (mais volumes EBS), justificável quando se requer elasticidade real e disponibilidade.

Facilidade de gerenciamento. O Minikube é simples e efêmero (criar/destruir em minutos), mas não representa produção. O EKS transfere à AWS a operação do plano de controle, ao custo de mais configuração inicial (rede/VPC, IAM, provedor de métricas, *registry*).

A Tabela VII consolida os **resultados empíricos** das duas execuções reais. Os números confirmam que o *alvo* de autoescalamento (1 a 10 réplicas, 50 % de CPU) foi atingido igualmente, mas com **tempo de subida menor no EKS** (dois nós, mais CPU agregada) e com a **distribuição de réplicas** que só a topologia multi-nó permite. A descida obedeceu à mesma janela de 300 s nas duas plataformas, reforçando que a assimetria temporal é intrínseca ao controlador.

Reprodutibilidade

Todos os artefatos — `Dockerfile` e código da aplicação, manifestos comentados, *scripts* de automação, roteiros passo a passo das duas implantações e as saídas coletadas — estão versionados em repositório público, permitindo que qualquer integrante do grupo reproduza os experimentos de forma independente. O roteiro da nuvem indica, etapa a etapa, quais recursos criar, quais evidências capturar e a ordem obrigatória de limpeza, minimizando o risco de recursos órfãos gerando custo.

VIII. DISCUSSÃO

Dois pontos merecem destaque. Primeiro, o **HPA escala Pods, não nós**: no Minikube de nó único, as dez réplicas couberam no mesmo nó porque havia CPU disponível (quatro vCPUs para um teto que exige, no máximo, dez vezes 100 m de CPU-alvo); em produção, atingir o teto de réplicas com alvos maiores pode exigir **mais nós**, o que o HPA *não* provê. Caberia ao *Cluster Autoscaler* ou ao Karpenter no EKS criar nós adicionais; sem eles, os Pods excedentes ficariam Pending. A combinação HPA + autoescalador de nós é, portanto, o que aproxima o comportamento de uma elasticidade fim a fim.

Segundo, o **custo** inverte a avaliação conforme o uso. Para validar o mecanismo, estudar e desenvolver, o Minikube é imbatível: gratuito, rápido de criar e destruir. Para operar uma aplicação real com elasticidade e disponibilidade, o Minikube não serve — não há múltiplos nós, zonas ou balanceador gerenciado —, e o EKS passa a compensar seu custo por hora. A escolha entre as plataformas não é, portanto, de “melhor” ou “pior”, mas de *propósito*: laboratório versus produção.

A **assimetria temporal** observada (subida rápida, descida lenta) é uma decisão de projeto do controlador que reflete uma preferência por disponibilidade sobre economia imediata: é preferível manter réplicas ociosas por alguns minutos a arriscar remover capacidade prematuramente diante de uma carga intermitente. Aplicações com padrões de carga muito irregulares podem ajustar as janelas de estabilização para equilibrar custo e disponibilidade.

Do ponto de vista **econômico**, o autoescalamento só se traduz em economia real quando acompanha a elasticidade da *infraestrutura*. No experimento, o HPA reduziu as réplicas de 10 para 1 ao cessar a carga, mas, no EKS, os **dois nós EC2 permaneceram provisionados** — e cobrados — independentemente do número de Pods. A economia de custo de computação só se materializaria com um autoescalador de nós (Cluster Autoscaler ou Karpenter) que desprovisionasse a capacidade ociosa após a redução das réplicas. Assim, o HPA é condição *necessária*, mas não *suficiente*, para elasticidade de custo: ele otimiza a alocação de réplicas sobre uma capacidade que precisa, ela mesma, ser elástica. Essa distinção — entre elasticidade de aplicação e elasticidade de infraestrutura — é central para dimensionar corretamente um ambiente de produção e evitar a ilusão de que “ligar o HPA” basta para reduzir a fatura.

IX. AMEAÇAS À VALIDADE

A carga é **sintética** e limitada por CPU; aplicações reais podem ser limitadas por memória, E/S ou latência de dependências, casos em que a métrica de CPU não capturaria a demanda (exigindo métricas customizadas). O experimento local usa **um único nó** com quatro vCPUs, o que limita a CPU total disponível e pode induzir *throttling* no pico, afetando a utilização medida. Os tempos derivam de amostragem a cada 20 s, com granularidade correspondente. Por fim, a comparação de custo é **qualitativa**; valores exatos dependem de região, tipo de instância e tempo de execução, a serem confirmados na calculadora de preços da AWS.

X. CONCLUSÃO

Demonstrou-se, de ponta a ponta, o autoescalamento horizontal de um servidor web em Kubernetes. Na implantação local com Minikube, o HPA reagiu a uma carga sintética escalando de 1 para 10 réplicas em cerca de três minutos (1→4→8→10) e retornando a uma réplica após a janela de estabilização de 300 s, comprovando empiricamente o mecanismo e sua dinâmica assimétrica (subida agressiva, descida conservadora). A implantação em Amazon EKS, com dois nós, reutilizou exatamente os mesmos manifestos e protocolo e

reproduziu a mesma escala (1 a 10) de forma ainda mais rápida (cerca de 1,5 min), com a diferença marcante de **distribuir as réplicas entre os nós** (5/5) — balanceamento impossível no nó único local. Conclui-se que o **mecanismo** de autoescalamento é portátil entre as plataformas, mas o **contexto** — custo, distribuição entre nós e a necessidade de escalar também a capacidade de nós — é o que efetivamente distingue um laboratório local de uma operação elástica em nuvem. Como trabalho futuro, propõe-se combinar o HPA com o *Cluster Autoscaler*/Karpenter no EKS, avaliar o autoescalamento sob métricas customizadas além da CPU e comparar o autoescalamento reativo com abordagens preditivas.

REFERÊNCIAS

- [1] Kubernetes Documentation, “Horizontal Pod Autoscaler Walkthrough”. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>.
- [2] AWS EKS Documentation, “Scale pod deployments with Horizontal Pod Autoscaler”. <https://docs.aws.amazon.com/eks/latest/userguide/horizontal-pod-autoscaler.html>.
- [3] The Kubernetes Authors, “Kubernetes Documentation: Concepts”. <https://kubernetes.io/docs/concepts/>.
- [4] Kubernetes SIGs, “Kubernetes Metrics Server”. <https://github.com/kubernetes-sigs/metrics-server>.